

第一周：GD 与 SGD 理论基石

——理论与工程练习完整解答——

上海财经大学·计算机机械社

理论练习

练习 1：下降引理的逐行推导

题目：若 $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ ，证明 $f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2$ 。

证明 (解答)： 设 $x, y \in \mathbb{R}^d$ 。考虑线段 $x + t(y - x)$ ($t \in [0, 1]$)，定义辅助函数 $h(t) = f(x + t(y - x))$ 。则 $h(0) = f(x)$ ， $h(1) = f(y)$ 。

由链式法则：

$$h'(t) = \langle \nabla f(x + t(y - x)), y - x \rangle. \quad (1)$$

应用微积分基本定理：

$$f(y) - f(x) = h(1) - h(0) = \int_0^1 h'(t) dt = \int_0^1 \langle \nabla f(x + t(y - x)), y - x \rangle dt. \quad (2)$$

将内积拆分为两项 (加减 $\nabla f(x)$)：

$$\begin{aligned} f(y) - f(x) &= \int_0^1 \langle \nabla f(x), y - x \rangle dt + \int_0^1 \langle \nabla f(x + t(y - x)) - \nabla f(x), y - x \rangle dt \\ &= \langle \nabla f(x), y - x \rangle + \int_0^1 \langle \nabla f(x + t(y - x)) - \nabla f(x), y - x \rangle dt. \end{aligned} \quad (3)$$

第一项不含 t ，故积分为简单的 $\langle \nabla f(x), y - x \rangle$ 。

对第二项应用 *Cauchy-Schwarz* 不等式：

$$|\langle \nabla f(x + t(y - x)) - \nabla f(x), y - x \rangle| \leq \|\nabla f(x + t(y - x)) - \nabla f(x)\| \cdot \|y - x\|. \quad (4)$$

利用 L -光滑性假设： $\|\nabla f(x + t(y - x)) - \nabla f(x)\| \leq L\|t(y - x)\| = Lt\|y - x\|$ 。代入：

$$|\langle \nabla f(x + t(y - x)) - \nabla f(x), y - x \rangle| \leq Lt\|y - x\|^2. \quad (5)$$

将上式代入式 (3) 的积分：

$$\begin{aligned} f(y) - f(x) &\leq \langle \nabla f(x), y - x \rangle + \int_0^1 Lt \|y - x\|^2 dt \\ &= \langle \nabla f(x), y - x \rangle + L \|y - x\|^2 \int_0^1 t dt \\ &= \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2. \end{aligned} \quad (6)$$

移项即得到下降引理：

$$\boxed{f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2.} \quad (7)$$

注记：该不等式将函数值 $f(y)$ 的上界表示为一个线性项（一阶近似）加上一个二次罚项（由光滑性常数 L 度量）。正是这个二次界保证了：只要步长 $\eta < 2/L$ ，梯度下降每步一定会使函数值下降。

练习 2：GD 在二次函数上的精确收敛因子

题目：考虑 $f(\theta) = \frac{1}{2}\theta^\top A\theta$, $A = \text{diag}(\lambda_1, \dots, \lambda_d)$ 。写出 GD 在第 i 个坐标上的递推，并推导最优步长 η 和收敛因子 $(\kappa - 1)/(\kappa + 1)$ 。

证明（解答）。第一步：按坐标解耦。

梯度为 $\nabla f(\theta) = A\theta = (\lambda_1\theta_1, \dots, \lambda_d\theta_d)^\top$ 。GD 更新 $\theta_{t+1} = \theta_t - \eta\nabla f(\theta_t)$ 在坐标 i 上化为：

$$\theta_{t+1,i} = \theta_{t,i} - \eta\lambda_i\theta_{t,i} = (1 - \eta\lambda_i)\theta_{t,i}. \quad (8)$$

递推 T 步：

$$\boxed{\theta_{T,i} = (1 - \eta\lambda_i)^T \theta_{0,i}.} \quad (9)$$

由于最优解为 $\theta^* = 0$ （因为 $f(\theta) = \frac{1}{2}\theta^\top A\theta \geq 0$ ，且 $A \succ 0$ ），误差由 $|1 - \eta\lambda_i|^T$ 控制。

第二步：最坏坐标的收敛因子。

总体收敛速度由衰减最慢的坐标决定。定义

$$\rho(\eta) = \max_{i=1,\dots,d} |1 - \eta\lambda_i|. \quad (10)$$

这是迭代矩阵 $I - \eta A$ 的谱半径。我们要找到 η 最小化 $\rho(\eta)$ 。

令 $\lambda_{\min} = \min_i \lambda_i$, $\lambda_{\max} = \max_i \lambda_i$, $\kappa = \lambda_{\max}/\lambda_{\min}$ 。函数 $|1 - \eta\lambda|$ 随 λ 的变化如下图：

- 当 $\eta\lambda < 1$ 时， $1 - \eta\lambda > 0$ 。
- 当 $\eta\lambda > 1$ 时， $1 - \eta\lambda < 0$ 。

$$\rho(\eta) = \max(|1 - \eta\lambda_{\min}|, |1 - \eta\lambda_{\max}|)。$$

第三步：求最优 η 。

考虑以下三种情况：

- 若 η 很小 ($\eta\lambda_{\max} < 1$)，则两项皆正， $\rho(\eta) = 1 - \eta\lambda_{\min}$ (最小值方向的衰减决定最坏情况)。
- 若 η 较大 ($\eta\lambda_{\min} > 1$)，则两项皆负， $\rho(\eta) = \eta\lambda_{\max} - 1$ (最大值方向决定最坏情况)。
- 最优 η 在两者交叉处达到：

$$1 - \eta\lambda_{\min} = \eta\lambda_{\max} - 1. \quad (11)$$

解出：

$$2 = \eta(\lambda_{\min} + \lambda_{\max}) \implies \boxed{\eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}}. \quad (12)$$

第四步：最优收敛因子。

代入最优步长 η^* ：

$$\begin{aligned} \rho(\eta^*) &= 1 - \eta^*\lambda_{\min} = 1 - \frac{2\lambda_{\min}}{\lambda_{\min} + \lambda_{\max}} \\ &= \frac{\lambda_{\min} + \lambda_{\max} - 2\lambda_{\min}}{\lambda_{\min} + \lambda_{\max}} \\ &= \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} \\ &= \boxed{\frac{\kappa - 1}{\kappa + 1}}. \end{aligned} \quad (13)$$

因此，经过 T 步 GD 后：

$$\boxed{\|\theta_T\|^2 \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^{2T} \|\theta_0\|^2}. \quad (14)$$

对应的函数值收敛：

$$f(\theta_T) \leq \frac{\lambda_{\max}}{2} \left(\frac{\kappa - 1}{\kappa + 1}\right)^{2T} \|\theta_0\|^2. \quad (15)$$

注记：当 $\eta = 1/\lambda_{\max}$ 时 (较保守的步长)，收敛因子为 $1 - 1/\kappa$ 。两种步长的对比：

$$\frac{\kappa - 1}{\kappa + 1} = 1 - \frac{2}{\kappa + 1} \quad \text{vs} \quad 1 - \frac{1}{\kappa}.$$

当 $\kappa \gg 1$ 时， $\frac{\kappa-1}{\kappa+1} \approx 1 - \frac{2}{\kappa}$ ，而 $1 - \frac{1}{\kappa} \approx 1 - \frac{1}{\kappa}$ ，前者后者的两倍快 (即其 log 衰减率约两倍)。

练习 3：条件数与梯度的几何

题目：研究 GD 在 $f(\theta) = \frac{1}{2}\theta^\top Q \text{diag}(1, \kappa)Q^\top \theta$ 上的轨迹。分别考虑 $Q = I$ 和 Q 为任意旋转的情况。

证明（解答）. 情况一： $Q = I$ （坐标对齐， $A = \text{diag}(1, \kappa)$ ）

设 $\kappa = 10$ 。梯度为 $\nabla f(\theta) = (\theta_1, \kappa\theta_2)^\top$ 。GD 更新：

$$\begin{pmatrix} \theta_1^{(t+1)} \\ \theta_2^{(t+1)} \end{pmatrix} = \begin{pmatrix} 1 - \eta & 0 \\ 0 & 1 - \kappa\eta \end{pmatrix} \begin{pmatrix} \theta_1^{(t)} \\ \theta_2^{(t)} \end{pmatrix}. \quad (16)$$

取 $\eta = 2/(1 + \kappa) = 2/11 \approx 0.182$ 。则：

$$\theta_1^{(t)} = \left(1 - \frac{2}{11}\right)^t \theta_1^{(0)} = \left(\frac{9}{11}\right)^t \theta_1^{(0)}, \quad (17)$$

$$\theta_2^{(t)} = \left(1 - \frac{20}{11}\right)^t \theta_2^{(0)} = \left(-\frac{9}{11}\right)^t \theta_2^{(0)}. \quad (18)$$

θ_1 缓慢衰减（方向 1 是「慢方向」，曲率小）， θ_2 快速衰减且符号翻转（方向 2 是「快方向」，曲率大）。轨迹在 (θ_1, θ_2) 平面中呈现「之字形」锯齿形下降，沿着慢方向（ θ_1 轴）缓慢爬向原点。

情况二： $Q \neq I$ （任意旋转）

设 $A = Q \text{diag}(1, \kappa)Q^\top$ ，其中 Q 是正交矩阵（ $QQ^\top = I$ ）。令 $y = Q^\top \theta$ （坐标变换到 *Hessian* 的特征基）。则：

$$f(\theta) = \frac{1}{2}\theta^\top A \theta = \frac{1}{2}\theta^\top Q \text{diag}(1, \kappa)Q^\top \theta = \frac{1}{2}y^\top \text{diag}(1, \kappa)y = \frac{1}{2}(y_1^2 + \kappa y_2^2). \quad (19)$$

GD 在 θ 空间的更新：

$$\theta_{t+1} = \theta_t - \eta A \theta_t = \theta_t - \eta Q \text{diag}(1, \kappa)Q^\top \theta_t. \quad (20)$$

在 y 空间中：

$$y_{t+1} = Q^\top \theta_{t+1} = Q^\top (\theta_t - \eta Q \text{diag}(1, \kappa)Q^\top \theta_t) = y_t - \eta \text{diag}(1, \kappa)y_t = (I - \eta \text{diag}(1, \kappa))y_t. \quad (21)$$

因此 y 空间中的递推与情况一完全一致！GD 的动力学在旋转下完全等价。

为什么 GD 是旋转不变的？

因为无约束 GD 的预条件器是 I （单位矩阵），且 $\|\cdot\|_2$ 在正交变换下不变。任意旋转只是重新标记了坐标轴，GD 的「下降方向 = 负梯度方向」这一几何事实不改变。

为什么对角预条件方法（如 AdaGrad / Adam）在旋转下表现不同？

对角预条件器 $\text{diag}(h_1, \dots, h_d)$ 只缩放坐标轴，不做旋转变换。若 $A = Q \text{diag}(1, \kappa)Q^\top$ 且 Q 不是置换矩阵，则对角预条件器 $\text{diag}(\sqrt{v_t})$ 并不能与 *Hessian* 的特征方向对齐。这意味着：

- 在 $Q = I$ 时, 对角预条件器可以完美适应 *Hessian* (只需对坐标 2 使用较小的学习率)。
- 在 $Q \neq I$ 时, 对角预条件器无法去除坐标间的耦合——它的效果等价于在原始空间中使用一个对角的正定矩阵 D 代替 I 作为预条件器, 即 $\theta_{t+1} \approx \theta_t - \eta D^{-1} \nabla f(\theta_t)$ 。这改变了优化几何 (从欧氏变为 *Mahalanobis*), 但可能不足以应对 *Hessian* 的非对角结构。

这是为什么 *Adam* 等自适应方法在全连接层 (特征维度间无自然排序) 上的表现有时不如 *SGD*, 而在嵌入/稀疏特征 (特征维度有自然意义且 *coordinate-aligned* 的对角 *Hessian* 近似成立) 上表现极佳。

练习 4: SGD 方差的实验测量

题目: 在二维二次函数上向梯度添加高斯噪声模拟 SGD, 研究最终误差与步长的关系。

证明 (理论与实验分析). 理论分析:

设 $f(\theta) = \frac{1}{2} \theta^\top A \theta$, $A \succ 0$ 。随机梯度 $g_t = A\theta_t + \xi_t$, 其中 $\xi_t \sim \mathcal{N}(0, \sigma^2 I)$ 。

由 SGD 在凸光滑条件下的分析 (见 *week1* 讲义定理 5: SGD 在凸光滑条件下的收敛率):

$$\mathbb{E}[f(\bar{\theta}_T)] - f(\theta^*) \leq \frac{\|\theta_0\|^2}{2\eta T} + \frac{\eta\sigma^2}{2}. \quad (22)$$

当 $T \rightarrow \infty$ 时, 第一项 $\rightarrow 0$, 但第二项 $\eta\sigma^2/2$ 不随 T 减小而消失。这就是 SGD 的稳态震荡:

$$\boxed{\lim_{T \rightarrow \infty} \mathbb{E}[f(\theta_T)] - f(\theta^*) \approx \frac{\eta\sigma^2}{2}}. \quad (23)$$

更精确的分析 (针对二次函数):

考虑一维情形 $f(\theta) = \frac{\lambda}{2} \theta^2$ 。SGD 更新:

$$\theta_{t+1} = \theta_t - \eta(\lambda\theta_t + \xi_t) = (1 - \eta\lambda)\theta_t - \eta\xi_t. \quad (24)$$

θ_t 是一个一阶自回归过程。当 $|1 - \eta\lambda| < 1$ (即 $\eta < 2/\lambda$) 时, θ_t 趋于平稳分布, 其平稳方差为:

$$(\theta_\infty) = \frac{\eta^2 \sigma^2}{1 - (1 - \eta\lambda)^2} = \frac{\eta\sigma^2}{\lambda(2 - \eta\lambda)}. \quad (25)$$

因此:

$$\mathbb{E}[f(\theta_\infty)] = \frac{\lambda}{2} \mathbb{E}[\theta_\infty^2] = \frac{\lambda}{2} \cdot \frac{\eta\sigma^2}{\lambda(2 - \eta\lambda)} = \frac{\eta\sigma^2}{2(2 - \eta\lambda)}. \quad (26)$$

取小步长 $\eta\lambda \ll 1$:

$$\mathbb{E}[f(\theta_\infty)] \approx \frac{\eta\sigma^2}{4}. \quad (27)$$

最优步长的 *trade-off*:

总误差（有限 T 下）由两项组成：

$$E(\eta) = \underbrace{\frac{\|\theta_0\|^2}{2\eta T}}_{\text{收敛项 (随 } \eta \text{ 增大而减小)}} + \underbrace{\frac{\eta\sigma^2}{2}}_{\text{稳态误差 (随 } \eta \text{ 增大而增大)}}. \quad (28)$$

令 $E'(\eta) = 0$:

$$-\frac{\|\theta_0\|^2}{2\eta^2 T} + \frac{\sigma^2}{2} = 0 \implies \boxed{\eta_{opt} = \frac{\|\theta_0\|}{\sigma\sqrt{T}}}. \quad (29)$$

此时最优误差：

$$\boxed{E(\eta_{opt}) = \frac{\|\theta_0\| \sigma}{\sqrt{T}}}. \quad (30)$$

这精确匹配了定理中的 $\mathcal{O}(1/\sqrt{T})$ 速率。

实验预期：

- 固定 $T = 1000$ ，变化 $\eta \in [10^{-3}, 1]$ 。
- η 很小时：收敛项大，误差大。
- η 在最优值附近：误差最小。
- η 较大时：稳态噪声主导，误差大。
- $\eta \geq 2/\lambda_{\max}$ 时：发散。

典型的「最终误差 vs η 」曲线呈 U 形，底部对应最优步长。

练习 5：Gradient Descent + Momentum 的特征分析

题目：在一维二次函数 $f(\theta) = \frac{\lambda}{2}\theta^2$ 上，分析 GD+Momentum 的状态空间方程，找出使谱半径最小化的 (η, β) 。

证明（解答）。第一步：建立状态空间方程。

一维二次函数： $\nabla f(\theta) = \lambda\theta$ 。

Momentum 更新（Polyak 重球法）：

$$\begin{aligned} d_t &= \beta d_{t-1} + \lambda\theta_t, \\ \theta_{t+1} &= \theta_t - \eta d_t. \end{aligned} \quad (31)$$

将 d_t 代入 θ_{t+1} 的表达式：

$$\theta_{t+1} = \theta_t - \eta(\beta d_{t-1} + \lambda\theta_t) = (1 - \eta\lambda)\theta_t - \eta\beta d_{t-1}. \quad (32)$$

定义状态向量 $x_t = (\theta_t, d_{t-1})^\top$ 。则：

$$\begin{aligned}\theta_{t+1} &= (1 - \eta\lambda)\theta_t - \eta\beta d_{t-1}, \\ d_t &= \beta d_{t-1} + \lambda\theta_t.\end{aligned}\tag{33}$$

写为矩阵形式：

$$\begin{pmatrix} \theta_{t+1} \\ d_t \end{pmatrix} = \underbrace{\begin{pmatrix} 1 - \eta\lambda & -\eta\beta \\ \lambda & \beta \end{pmatrix}}_M \begin{pmatrix} \theta_t \\ d_{t-1} \end{pmatrix}.\tag{34}$$

第二步：求特征值。

矩阵 M 的特征多项式：

$$\det(M - \mu I) = \begin{vmatrix} 1 - \eta\lambda - \mu & -\eta\beta \\ \lambda & \beta - \mu \end{vmatrix} = 0.\tag{35}$$

展开：

$$(1 - \eta\lambda - \mu)(\beta - \mu) - (-\eta\beta)(\lambda) = 0.\tag{36}$$

$$(1 - \eta\lambda - \mu)(\beta - \mu) + \eta\beta\lambda = 0.\tag{37}$$

$$\mu^2 - (1 - \eta\lambda + \beta)\mu + \beta(1 - \eta\lambda) + \eta\beta\lambda = 0.\tag{38}$$

$$\mu^2 - (1 - \eta\lambda + \beta)\mu + \beta = 0.\tag{39}$$

两个特征值为：

$$\mu_{1,2} = \frac{(1 - \eta\lambda + \beta) \pm \sqrt{(1 - \eta\lambda + \beta)^2 - 4\beta}}{2}.\tag{40}$$

第三步：最小化谱半径。

谱半径 $\rho(M) = \max(|\mu_1|, |\mu_2|)$ 。我们希望最小化 $\rho(M)$ 。

有两种情况：

- 实特征值（判别式 ≥ 0 ）： $\mu_{1,2}$ 皆为正实数（假设 $\beta \geq 0, \eta$ 适当），则 $\rho(M) = \mu_{\max}$ 。
- 复特征值（判别式 < 0 ）： $\mu_{1,2}$ 为共轭复数，则 $|\mu_1| = |\mu_2| = \sqrt{\beta}$ 。

在复特征值情形下，谱半径为 $\sqrt{\beta}$ ，与 η 无关（只要判别式 < 0 的条件成立）。这意味著只要步长选在合适的区间内，收敛速度只由动量系数 β 决定。

判别式 < 0 的条件：

$$(1 - \eta\lambda + \beta)^2 < 4\beta.\tag{41}$$

即：

$$-2\sqrt{\beta} < 1 - \eta\lambda + \beta < 2\sqrt{\beta}.\tag{42}$$

整理关于 $\eta\lambda$ 的条件：

$$(1 - \sqrt{\beta})^2 < \eta\lambda < (1 + \sqrt{\beta})^2. \quad (43)$$

在此区间内， $\rho(M) = \sqrt{\beta}$ 。我们可以选择 η 使判别式为零（临界阻尼临界点，此时收敛最快）：

$$(1 - \eta\lambda + \beta)^2 = 4\beta \implies \eta\lambda = (1 - \sqrt{\beta})^2. \quad (44)$$

或 $\eta\lambda = (1 + \sqrt{\beta})^2$ （后者对应的 η 更大，通常不选）。

取 $\eta\lambda = (1 - \sqrt{\beta})^2$ ，此实为「临界复特征值」的边界。

第四步：推广到多维和加速因子。

对于多维二次函数 $f(\theta) = \frac{1}{2}\theta^\top A\theta$ ，在 A 的特征基下解耦为 d 个独立的过程。令 $\kappa = \lambda_{\max}/\lambda_{\min}$ 。最优参数选择为：

$$\eta^* = \frac{4}{(\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}})^2}, \quad \beta^* = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^2. \quad (45)$$

在此参数下，收敛因子（谱半径）为：

$$\rho^* = \sqrt{\beta^*} = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}. \quad (46)$$

对比：GD vs Momentum 的收敛因子

方法	收敛因子（每步）	$\kappa = 100$ 时的数值
GD ($\eta = 1/\lambda_{\max}$)	$1 - \frac{1}{\kappa}$	0.99
GD ($\eta = 2/(\lambda_{\min} + \lambda_{\max})$)	$\frac{\kappa-1}{\kappa+1}$	0.9802
Momentum（最优参数）	$\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$	0.8182

加速因子的定量解释：

当 $\kappa \gg 1$ 时：

$$\rho_{GD} = \frac{\kappa - 1}{\kappa + 1} \approx 1 - \frac{2}{\kappa}, \quad (47)$$

$$\rho_{Momentum} = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \approx 1 - \frac{2}{\sqrt{\kappa}}. \quad (48)$$

达到误差 ε 所需的迭代数：

$$T_{GD} \approx \frac{\kappa}{2} \log \frac{1}{\varepsilon}, \quad (49)$$

$$T_{Momentum} \approx \frac{\sqrt{\kappa}}{2} \log \frac{1}{\varepsilon}. \quad (50)$$

当 $\kappa = 10000$ 时，Momentum 将迭代数减少约 $100\times$ 。这就是「加速」的含义。

工程练习

工程任务 1：实现并可视化 GD

目标：在 $\kappa = 100$ 的二次函数上验证 GD 的收敛行为。

Listing 1: GD 在不同步长下的收敛行为

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ---- 问题定义 ----
5 d = 2
6 A = np.diag([1.0, 100.0])          # kappa = 100
7 theta0 = np.array([10.0, 1.0])    # 初始点
8 theta_star = np.zeros(d)          # 最优解
9
10 def f(theta):
11     return 0.5 * theta @ A @ theta
12
13 def grad_f(theta):
14     return A @ theta
15
16 # ---- GD 运行 ----
17 L = np.max(np.diag(A))             # lambda_max = 100
18 etas = [0.0001, 0.001, 0.01, 0.019, 0.02, 0.021]
19 T = 200
20
21 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
22
23 for eta in etas:
24     theta = theta0.copy()
25     losses = []
26     thetas = [theta.copy()]
27
28     for t in range(T):
29         g = grad_f(theta)
30         theta = theta - eta * g
31         losses.append(f(theta))
32         thetas.append(theta.copy())
33
```

```

34 # 左图: loss curve
35 factor = eta * L
36 label = f"$\\eta={eta}$, $\\eta L={factor:.2f}$"
37 ax1.semilogy(losses, label=label, alpha=0.8)
38 ax1.axhline(1e-15, color='gray', ls='--', alpha=0.3)
39
40 # 右图: trajectory (只画不发散的)
41 thetas = np.array(thetas)
42 if losses[-1] < 1e5: # 未发散
43     ax2.plot(thetas[:, 0], thetas[:, 1], '.-',
44             alpha=0.5, markersize=2, label=label)
45
46 ax1.set_xlabel('迭代次数')
47 ax1.set_ylabel('$f(\\theta_t)$ (log scale)')
48 ax1.set_title('GD 在不同步长下的收敛 ($\\kappa=100$)')
49 ax1.legend(fontsize=7, loc='lower left')
50 ax1.grid(True, alpha=0.3)
51
52 ax2.set_xlabel('$\\theta_1$')
53 ax2.set_ylabel('$\\theta_2$')
54 ax2.set_title('GD 轨迹 ($\\kappa=100$)')
55 ax2.legend(fontsize=7)
56 ax2.grid(True, alpha=0.3)
57 ax2.set_xlim(-12, 12)
58 ax2.set_ylim(-1.5, 1.5)
59
60 plt.tight_layout()
61 plt.savefig('task1_gd_convergence.pdf', dpi=150)
62 plt.show()

```

关键观察：

- $\eta L < 1$ 时单调下降； $\eta L = 1$ 时最快下降； $\eta L > 2$ 时发散。
- 轨迹图中，GD 在慢方向（ θ_1 轴）进展缓慢，在快方向（ θ_2 轴）迅速收敛。
- 由于 $\kappa = 100$ ， θ_1 的衰减因子约为 $1 - \eta$ ，而 θ_2 的衰减因子约为 $1 - 100\eta$ 。在 $\eta = 1/L = 0.01$ 时，前者为 0.99（极慢），后者为 0（一步收敛）——这正是「之字形」轨迹的数学原因。
- 最优步长 $\eta = 2/(1+100) \approx 0.0198$ 时，两个方向的衰减因子绝对值相等（ $99/101 \approx 0.98$ ），但代价是 θ_2 方向会产生符号翻转（overshoot）。

工程任务 2：SGD 的噪声与方差

目标：在二次函数上验证 SGD 的 $\mathcal{O}(1/\sqrt{T})$ 收敛率。

Listing 2: SGD 噪声方差与收敛率验证

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ---- 问题定义 ----
5 A = np.diag([1.0, 100.0])
6 theta0 = np.array([10.0, 1.0])
7
8 def f(theta):
9     return 0.5 * theta @ A @ theta
10
11 def grad_f(theta):
12     return A @ theta
13
14 # ---- (A) 不同 sigma 下的 SGD ----
15 eta = 1.0 / 100.0 # 固定步长 eta = 1/L
16 sigmas = [0.1, 1.0, 3.0]
17 T = 500
18 n_runs = 10
19 rng = np.random.RandomState(42)
20
21 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
22 ax1, ax2 = axes
23
24 colors = ['blue', 'orange', 'red']
25
26 for si, sigma in enumerate(sigmas):
27     all_losses = np.zeros((n_runs, T))
28
29     for run in range(n_runs):
30         theta = theta0.copy()
31         for t in range(T):
32             noise = sigma * rng.randn(2)
33             g = grad_f(theta) + noise
34             theta = theta - eta * g
35             all_losses[run, t] = f(theta)
```

```
36
37 # 均值与标准差带
38 mean_loss = all_losses.mean(axis=0)
39 std_loss = all_losses.std(axis=0)
40 ax1.semilogy(mean_loss, color=colors[si],
41              label=f'$\\sigma={sigma}$')
42 ax1.fill_between(range(T),
43                  np.maximum(mean_loss - std_loss, 1e-15),
44                  mean_loss + std_loss,
45                  alpha=0.2, color=colors[si])
46
47 # 加 GD 对照
48 theta = theta0.copy()
49 gd_losses = []
50 for t in range(T):
51     g = grad_f(theta)
52     theta = theta - eta * g
53     gd_losses.append(f(theta))
54 ax1.semilogy(gd_losses, 'k--', label='GD (无噪声)', alpha=0.8)
55
56 ax1.set_xlabel('迭代次数 $t$')
57 ax1.set_ylabel('$f(\\theta_t)$ (log scale)')
58 ax1.set_title('SGD 在不同噪声水平下的收敛')
59 ax1.legend(fontsize=9)
60 ax1.grid(True, alpha=0.3)
61
62 # ---- (B) 收敛率验证: log(error) vs log(T) ----
63 T_large = 5000
64 eta = 0.01
65 sigma = 1.0
66
67 theta = theta0.copy()
68 errors = []
69 for t in range(T_large):
70     noise = sigma * rng.randn(2)
71     g = grad_f(theta) + noise
72     theta = theta - eta * g
73     if (t + 1) % 50 == 0:
74         errors.append((t + 1, f(theta)))
75
```

```

76 errors = np.array(errors)
77 ax2.loglog(errors[:, 0], errors[:, 1], 'b-', alpha=0.8,
78             label='SGD ( $\eta=0.01$ ,  $\sigma=1$ )')
79
80 # 参考线:  $O(1/\sqrt{T})$  和  $O(1/T)$ 
81 t_vals = errors[:, 0]
82 ref_sqrt = 10 / np.sqrt(t_vals)      #  $O(1/\sqrt{T})$ 
83 ref_linear = 100 / t_vals            #  $O(1/T)$ 
84
85 ax2.loglog(t_vals, ref_sqrt, 'r--', alpha=0.7, label=' $O(1/\sqrt{T})$ ')
86 ax2.loglog(t_vals, ref_linear, 'g--', alpha=0.7, label=' $O(1/T)$ ')
87
88 ax2.set_xlabel('迭代次数  $T$  (log scale)')
89 ax2.set_ylabel(' $f(\theta_T)$  (log scale)')
90 ax2.set_title('SGD 收敛率验证')
91 ax2.legend(fontsize=9)
92 ax2.grid(True, alpha=0.3)
93
94 plt.tight_layout()
95 plt.savefig('task2_sgd_noise.pdf', dpi=150)
96 plt.show()
97
98 # ---- (C) 稳态误差 vs eta ----
99 sigmas_test = [0.5, 1.0, 2.0]
100 etas_test = np.logspace(-3, -0.3, 30)
101 T_warm = 5000
102
103 fig3, ax3 = plt.subplots(figsize=(8, 5))
104
105 for sigma in sigmas_test:
106     final_errors = []
107     for eta in etas_test:
108         # 检查稳定性
109         if eta >= 2.0 / 100.0:
110             final_errors.append(np.nan)
111             continue
112         theta = np.zeros(2)
113         # 预热

```

```

114     for t in range(T_warm):
115         noise = sigma * rng.randn(2)
116         g = grad_f(theta) + noise
117         theta = theta - eta * g
118     # 测量稳态
119     steady = 0.0
120     for t in range(500):
121         noise = sigma * rng.randn(2)
122         g = grad_f(theta) + noise
123         theta = theta - eta * g
124         steady += f(theta)
125     final_errors.append(steady / 500)
126
127     final_errors = np.array(final_errors)
128     valid = ~np.isnan(final_errors)
129     ax3.loglog(etas_test[valid], final_errors[valid], 'o-',
130               label=f'$\\sigma={sigma}$', alpha=0.8)
131
132     # 理论线: eta * sigma^2 / 4
133     theory = etas_test * sigma**2 / 4
134     ax3.loglog(etas_test, theory, '--', alpha=0.4,
135               label=f'理论: $\\eta\\sigma^2/4$' if sigma ==
136                   sigmas_test[0] else '')
137
138     ax3.set_xlabel('步长 $\\eta$ (log scale)')
139     ax3.set_ylabel('稳态误差 (log scale)')
140     ax3.set_title('SGD 稳态误差 vs 步长 (验证 $\\propto \\eta\\sigma^2$)')
141
142     plt.tight_layout()
143     plt.savefig('task2_steady_state.pdf', dpi=150)
144     plt.show()
145
146     print("=== 关键发现 ===")
147     print("1. SGD 的误差在 log-log 图上的斜率约为 -0.5, 验证了  $O(1/\sqrt{T})$  速率。")
148     print("2. 噪声越大 (sigma 大), 收敛越慢且稳态误差越高。")
149     print("3. 稳态误差正比于  $\eta * \sigma^2$ , 与理论一致。")
150

```

关键观察：

- 左图展示了不同 σ 下 SGD 的均值和标准差带。 σ 越大，收敛后的震荡幅度越大。
- 右图 (log-log) 中 SGD 的斜率约为 $-1/2$ ，而 GD 的参考线斜率为 -1 ——直观验证了 SGD 的慢 $\sqrt{T}$ 因子。
- 最终误差 vs η 的曲线验证了理论：在小 η 时受收敛速度限制，在大 η 时受稳态噪声限制，存在一个最优 η 平衡两者。

工程任务 3：Momentum 与 Nesterov 的加速效果

目标：在 $\kappa = 1000$ 的二次函数上比较 GD、Momentum、Nesterov。

Listing 3: GD vs Momentum vs Nesterov 加速对比

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ---- 问题定义 ----
5 kappa = 1000
6 A = np.diag([1.0, kappa])          # kappa = 1000
7 theta0 = np.array([10.0, 1.0])
8 theta_star = np.zeros(2)
9
10 def f(theta):
11     return 0.5 * theta @ A @ theta
12
13 def grad_f(theta):
14     return A @ theta
15
16 lambda_min = 1.0
17 lambda_max = kappa
18 L = lambda_max
19 mu = lambda_min
20
21 # ---- 最优参数 ----
22 # GD: eta = 2/(lambda_min + lambda_max)
23 eta_gd = 2.0 / (lambda_min + lambda_max)
24
25 # Momentum (Polyak): 最优参数
```

```
26 beta_momentum = ((np.sqrt(kappa) - 1) / (np.sqrt(kappa) + 1))**2
27 eta_momentum = (2 - 2 * beta_momentum) / lambda_max
28
29 # Nesterov: 标准参数
30 eta_nest = 1.0 / L
31 beta_nest = (np.sqrt(kappa) - 1) / (np.sqrt(kappa) + 1)
32
33 T = 2000
34
35 # ---- 运行三种优化器 ----
36 def run_gd(theta0, eta, T):
37     theta = theta0.copy()
38     losses, thetas = [], [theta.copy()]
39     for t in range(T):
40         theta = theta - eta * grad_f(theta)
41         losses.append(f(theta))
42         thetas.append(theta.copy())
43     return np.array(losses), np.array(thetas)
44
45 def run_momentum(theta0, eta, beta, T):
46     theta = theta0.copy()
47     d = np.zeros_like(theta)
48     losses, thetas = [], [theta.copy()]
49     for t in range(T):
50         g = grad_f(theta)
51         d = beta * d + g
52         theta = theta - eta * d
53         losses.append(f(theta))
54         thetas.append(theta.copy())
55     return np.array(losses), np.array(thetas)
56
57 def run_nesterov(theta0, eta, beta, T):
58     theta = theta0.copy()
59     d = np.zeros_like(theta)
60     losses, thetas = [], [theta.copy()]
61     for t in range(T):
62         # Nesterov: 在前瞻点计算梯度
63         lookahead = theta - eta * beta * d
64         g = grad_f(lookahead)
65         d = beta * d + g
```



```

66     theta = theta - eta * d
67     losses.append(f(theta))
68     thetas.append(theta.copy())
69     return np.array(losses), np.array(thetas)
70
71 loss_gd, traj_gd = run_gd(theta0, eta_gd, T)
72 loss_mom, traj_mom = run_momentum(theta0, eta_momentum, beta_momentum
73     , T)
74 loss_nest, traj_nest = run_nesterov(theta0, eta_nest, beta_nest, T)
75
76 # ---- 绘图 ----
77
78 # 左图：收敛曲线
79 ax1 = axes[0]
80 ax1.semilogy(loss_gd, 'b-', alpha=0.8, label=f'GD ( $\eta=2/(\mu+L)$ )')
81 ax1.semilogy(loss_mom, 'orange', alpha=0.8,
82     label=f'Momentum ( $\beta=\{beta\_momentum:.3f\}$ )')
83 ax1.semilogy(loss_nest, 'g-', alpha=0.8,
84     label=f'Nesterov ( $\beta=\{beta\_nest:.3f\}$ )')
85
86 # 理论衰减参考线
87 t = np.arange(T)
88 rho_gd = (kappa - 1) / (kappa + 1)
89 rho_acc = (np.sqrt(kappa) - 1) / (np.sqrt(kappa) + 1)
90 ref_gd = f(theta0) * rho_gd**(2*t)
91 ref_acc = f(theta0) * rho_acc**t
92 ax1.semilogy(ref_gd, 'b--', alpha=0.3, label=f'理论：GD  $O((\frac{kappa-1}{kappa+1})^{2T})$ ')
93 ax1.semilogy(ref_acc, 'r--', alpha=0.3, label=f'理论：Accel  $O((\frac{\sqrt{kappa}-1}{\sqrt{kappa}+1})^T)$ ')
94
95 ax1.set_xlabel('迭代次数')
96 ax1.set_ylabel('$f(\theta_t)$ (log scale)')
97 ax1.set_title(f'GD vs Momentum vs Nesterov ( $\kappa=\{kappa\}$ )')
98 ax1.legend(fontsize=8)
99 ax1.grid(True, alpha=0.3)
100
101 # 右图：轨迹

```

```

102 ax2 = axes[1]
103 labels = [('GD', traj_gd, 'b'), ('Momentum', traj_mom, 'orange'),
104           ('Nesterov', traj_nest, 'g')]
105 for name, traj, color in labels:
106     ax2.plot(traj[:80, 0], traj[:80, 1], '-', color=color,
107             alpha=0.7, linewidth=1, label=name)
108     ax2.plot(traj[0, 0], traj[0, 1], 'o', color=color, markersize=6)
109
110 ax2.plot(0, 0, 'r*', markersize=12, label='Optimum')
111 ax2.set_xlabel('$\\theta_1$')
112 ax2.set_ylabel('$\\theta_2$')
113 ax2.set_title(f'前 80 步轨迹对比 ($\\kappa={kappa}$)')
114 ax2.legend(fontsize=8)
115 ax2.grid(True, alpha=0.3)
116
117 plt.tight_layout()
118 plt.savefig('task3_acceleration.pdf', dpi=150)
119 plt.show()
120
121 # ---- 加速因子验证 ----
122 tol = 1e-10
123 for name, losses in [('GD', loss_gd), ('Momentum', loss_mom),
124                     ('Nesterov', loss_nest)]:
125     steps = np.where(losses < tol)[0]
126     if len(steps) > 0:
127         print(f"{name}: 达到 {tol:.0e} 需要 {steps[0]} 步")
128     else:
129         print(f"{name}: 在 {T} 步内未达到 {tol:.0e}")
130
131 print(f"\n理论加速比: sqrt(kappa) = {np.sqrt(kappa):.1f}")
132 print(f"GD 理论收敛因子: {rho_gd:.6f}")
133 print(f"加速理论收敛因子: {rho_acc:.6f}")

```

关键观察：

- GD 的衰减因子约为 $1 - 2/\kappa = 0.998$ （极慢！），2000 步后误差仅下降约 $2\times$ 。
- Momentum 和 Nesterov 的衰减因子约为 $1 - 2/\sqrt{\kappa} \approx 0.939$ ，相同迭代数下的误差下降远快于 GD。
- 轨迹图（右）展示 Momentum/Nesterov 的「过冲」现象——沿快方向（ θ_2 轴）越

过最优点后折返，与 GD 沿慢方向的「之字形」形成鲜明对比。Momentum 利用了惯性在快方向上做了更大的步长并快速定位到谷底。

- Nesterov 方法在理论上与 Momentum 有相同的渐近速率，但「前瞻梯度」减少了过冲振幅，在实际中通常比标准 Momentum 稍快且更稳定。

工程任务 4：学习率调度对比

目标：在 logistic regression 上比较不同的学习率调度策略。

Listing 4: 学习率调度策略对比

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # ---- 生成数据 ----
5 np.random.seed(42)
6 n, d = 2000, 50
7 X = np.random.randn(n, d)
8 theta_true = np.random.randn(d)
9 # logit:  $p(y=1|x) = \text{sigmoid}(x^T \theta_{\text{true}})$ 
10 prob = 1.0 / (1.0 + np.exp(-X @ theta_true))
11 y = (np.random.rand(n) < prob).astype(np.float64)
12
13 def sigmoid(z):
14     return 1.0 / (1.0 + np.exp(-np.clip(z, -50, 50)))
15
16 def logistic_loss(theta, X, y):
17     p = sigmoid(X @ theta)
18     eps = 1e-12
19     return -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p +
20         eps))
21
22 def logistic_grad(theta, X, y):
23     return X.T @ (sigmoid(X @ theta) - y) / n
24
25 def sgd_with_schedule(theta0, X, y, T, lr_schedule):
26     """运行 SGD 并返回 loss 历史。lr_schedule(t) 返回第 t 步的学习
27     率。"""
28     theta = theta0.copy()
29     losses = []
30     rng = np.random.RandomState(123)
```

```
29     for t in range(T):
30         # mini-batch
31         idx = rng.choice(n, size=32, replace=False)
32         g = logistic_grad(theta, X[idx], y[idx])
33         eta = lr_schedule(t)
34         theta = theta - eta * g
35         if t % 20 == 0:
36             losses.append((t, logistic_loss(theta, X, y)))
37     return np.array(losses)
38
39 # ---- 调度定义 ----
40 T_total = 3000
41
42 # 1. 固定步长
43 def fixed(eta0=1.0):
44     return lambda t: eta0
45
46 # 2. Warmup + Cosine Decay
47 def warmup_cosine(eta_max=1.0, eta_min=1e-3, Tw=300, T=T_total):
48     def schedule(t):
49         if t < Tw:
50             return eta_max * t / Tw
51         progress = (t - Tw) / (T - Tw)
52         return eta_min + (eta_max - eta_min) * 0.5 * (1 + np.cos(np.
53             pi * progress))
54     return schedule
55
56 # 3. Step Decay
57 def step_decay(eta0=1.0, gamma=0.5, step_size=600):
58     def schedule(t):
59         return eta0 * (gamma ** (t // step_size))
60     return schedule
61
62 # 4. 多项式衰减
63 def poly_decay(eta0=1.0, p=2.0, T=T_total):
64     def schedule(t):
65         return eta0 * (1 - t / T) ** p
66     return schedule
67
68 # ---- 运行实验 ----
```

```
68 theta0 = np.zeros(d)
69 schedules = {
70     'Fixed ($\\eta=1.0$)': fixed(1.0),
71     'Fixed ($\\eta=0.1$)': fixed(0.1),
72     'Warmup+Cosine': warmup_cosine(),
73     'Step Decay ($\\gamma=0.5$)': step_decay(),
74     'Poly Decay ($p=2$)': poly_decay(),
75 }
76
77 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
78 colors = plt.cm.tab10(np.linspace(0, 1, len(schedules)))
79
80 for (name, lr_fn), color in zip(schedules.items(), colors):
81     result = sgd_with_schedule(theta0, X, y, T_total, lr_fn)
82     ax1.semilogy(result[:, 0], result[:, 1], color=color,
83                 label=name, alpha=0.8)
84
85     # 画学习率曲线
86     ts = np.arange(T_total)
87     lrs = [lr_fn(t) for t in ts]
88     ax2.plot(ts, lrs, color=color, label=name, alpha=0.8)
89
90 ax1.set_xlabel('迭代次数')
91 ax1.set_ylabel('Loss (log scale)')
92 ax1.set_title('不同学习率调度下的 SGD 收敛')
93 ax1.legend(fontsize=7)
94 ax1.grid(True, alpha=0.3)
95
96 ax2.set_xlabel('迭代次数')
97 ax2.set_ylabel('学习率 $\\eta_t$')
98 ax2.set_title('学习率调度曲线')
99 ax2.legend(fontsize=7)
100 ax2.grid(True, alpha=0.3)
101
102 plt.tight_layout()
103 plt.savefig('task4_lr_schedule.pdf', dpi=150)
104 plt.show()
105
106 # ---- 最终 loss 统计 ----
107 print("=== 最终 Loss (3000 步) ===")
```

```
108 for name, lr_fn in schedules.items():
109     res = sgd_with_schedule(theta0, X, y, T_total, lr_fn)
110     print(f"{name:30s}: {res[-1, 1]:.6f}")
```

关键观察：

- **固定步长 $\eta = 1.0$** ：初期进展快，但后期在最优解附近剧烈震荡，最终 loss 较高。
- **固定步长 $\eta = 0.1$** ：初期进展慢，但最终收敛到较低水平——体现了收敛速度与稳态精度的 trade-off。
- **Warmup+Cosine**：综合了两者的优点——早期有足够的步长快速进展，后期逐步衰减实现精准收敛。通常在实践中表现最佳。
- **Step Decay**：在每个衰减点出现「阶梯式」的 loss 下降，体现为 loss 曲线上的不连续跳跃。这种方法简单但需要预先指定衰减时间点，对问题敏感。
- **多项式衰减 ($p = 2$)**：平滑地将学习率递减到 0，最终精度较高，但衰减速度可能过快导致训练早期结束。

最佳实践建议：在深度学习训练中，Warmup + Cosine Decay 是当前最广泛使用的组合。Warmup 防止训练初期的不稳定，Cosine decay 提供了平滑且有效的学习率衰减。

总结：第一周核心收获

- (1) **下降引理**是所有 GD 收敛分析的起点——它利用 L -光滑性将函数值下降与梯度范数联系起来。
- (2) **条件数 $\kappa = L/\mu$** 直接决定了 GD 在强凸函数上的收敛速度。GD 在 κ 很大时极度缓慢，这是所有后续加速方法存在的根本原因。
- (3) **SGD 的 $\mathcal{O}(1/\sqrt{T})$ 速率**源于随机梯度的方差 σ^2 。方差越大，最优步长越小，收敛越慢。
- (4) **Momentum/Nesterov 加速**将有效条件数从 κ 降为 $\sqrt{\kappa}$ ，其本质是通过历史梯度的指数移动平均「放大」低频（持续下降的）方向的分量。
- (5) **学习率调度**是 SGD 在实践中不可或缺的组件：Warmup 防止初期发散，衰减策略保证后期精确收敛。